

Coding & Solution

ComicCraft - AI Comic Story Creator using Gemini Models

Madras Institute of Technology | Department of Computer Technology (CT) | III Year, V Semester

Project period: 1 Oct 2026 - 2 Oct 2026

Core source files are reproduced below (see the repository for the full project).

app/routes.py

```
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

from fastapi import APIRouter, Form, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel

from app.exporters import save_pdf
from app.gemini_flash import generate_outline
from app.gemini_pro import generate_story
from app.image_generator import generate_image
from app.layout_builder import build_comic_layout

router = APIRouter()
templates = Jinja2Templates(directory=Path(__file__).resolve().parent.parent / "templates")

class PromptRequest(BaseModel):
    prompt: str
    character: str
    setting: str
    tone: str
    style: str

def create_comic(prompt: str, character: str, setting: str, tone: str, style: str) -> tuple[list[dict], str]:
    try:
        outline = generate_outline(prompt, character, setting, tone, style)
        with ThreadPoolExecutor() as pool: # story text overlaps with image generation
            story_job = pool.submit(generate_story, outline, character, tone)
            images = [generate_image(p["image_prompt"]) for p in outline]
            story = story_job.result()
        layout = build_comic_layout(outline, images, story)
        return layout, save_pdf(layout)
    except Exception as exc:
        raise HTTPException(status_code=500, detail=f"Comic generation failed: {exc}") from exc

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(request, "index.html")

@router.post("/generate", response_class=HTMLResponse)
def generate(
    request: Request,
    prompt: str = Form(...),
    character: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    style: str = Form(...),
):
```

```

layout, pdf_path = create_comic(prompt, character, setting, tone, style)
return templates.TemplateResponse(request, "comic_preview.html", {"layout": layout, "pdf_path": pdf_path})

```

```

@router.post("/generate-comic/json")
def generate_json(body: PromptRequest):
    layout, pdf_path = create_comic(body.prompt, body.character, body.setting, body.tone, body.style)
    return {"layout": layout, "pdf_path": pdf_path}

```

```

@router.get("/export-success", response_class=HTMLResponse)
def export_success(request: Request, pdf: str = ""):
    return templates.TemplateResponse(request, "export_success.html", {"pdf_path": pdf})

```

```

@router.get("/test-image")
def test_image(prompt: str = "a brave fox in an enchanted forest"):
    try:
        return {"image": generate_image(prompt)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=f"Image generation failed: {exc}") from exc

```

app/gemini_flash.py

```

import json
import os

import google.generativeai as genai

MODEL = os.getenv("GEMINI_FLASH_MODEL", "models/gemini-1.5-flash")
PANELS = 5

def generate_outline(prompt: str, character: str, setting: str, tone: str, style: str) -> list[dict]:
    """Return a structured 5-panel outline: panel, title, scene, image_prompt."""
    genai.configure(api_key=os.getenv["GEMINI_API_KEY"])
    model = genai.GenerativeModel(
        MODEL, generation_config={"response_mime_type": "application/json"}
    )
    request = (
        f"Create a {PANELS}-panel comic outline.\n"
        f"Story idea: {prompt}\nMain character: {character}\nSetting: {setting}\n"
        f"Tone: {tone}\nArt style: {style}\n\n"
        f"Return a JSON array of exactly {PANELS} objects with keys: "
        f'"panel" (integer starting at 1), "title", "scene" (one-paragraph scene description), '
        f'"image_prompt" (a vivid image-generation prompt in {style} style featuring {character}).'
    )
    outline = json.loads(model.generate_content(request).text)
    if not isinstance(outline, list) or not outline:
        raise ValueError("Gemini Flash returned an invalid outline")
    return [
        {
            "panel": i,
            "title": str(p.get("title", f"Panel {i}")),
            "scene": str(p.get("scene", "")),
            "image_prompt": str(p.get("image_prompt", p.get("scene", ""))),
        }
        for i, p in enumerate(outline[:PANELS], 1)
    ]

```

app/gemini_pro.py

```

import json
import os

import google.generativeai as genai

```

```

MODEL = os.getenv("GEMINI_PRO_MODEL", "models/gemini-1.5-pro")

def generate_story(outline: list[dict], character: str, tone: str) -> str:
    """Expand the outline into narration and dialogue; one text block, panels delimited by 'Panel N:'. """
    genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
    model = genai.GenerativeModel(MODEL)
    request = (
        f"Write a {tone} comic story featuring {character} from this panel outline:\n"
        f"{json.dumps(outline, indent=2)}\n\n"
        "For every panel use exactly this plain-text format (no markdown):\n"
        "Panel <number>:\n"
        "Caption: <one short ambient line>\n"
        "Narration: <2-4 sentences of action, emotion and character dialogue>\n"
    )
    return model.generate_content(request).text

```

app/image_generator.py

```

import re
from pathlib import Path

PANELS_DIR = Path(__file__).resolve().parent.parent / "static" / "panels"
MODEL_ID = "runwayml/stable-diffusion-v1-5"

_pipe = None

def _get_pipe():
    global _pipe
    if _pipe is None:
        import torch
        from diffusers import StableDiffusionPipeline

        device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
        dtype = torch.float32 if device == "cpu" else torch.float16
        _pipe = StableDiffusionPipeline.from_pretrained(MODEL_ID, torch_dtype=dtype).to(device)
        _pipe.enable_attention_slicing()
    return _pipe

def generate_image(prompt: str) -> str:
    """Generate a comic-style image, save to static/panels, return its URL path. """
    PANELS_DIR.mkdir(parents=True, exist_ok=True)
    safe = re.sub(r"[^a-zA-Z0-9]+", "_", prompt).strip("_")[:60] or "panel"
    path = PANELS_DIR / f"{safe}.png"
    image = _get_pipe()(prompt + ", comic book illustration, detailed", num_inference_steps=25).images[0]
    image.save(path)
    return f"/static/panels/{path.name}"

```

app/layout_builder.py

```

import re

def build_comic_layout(outline: list[dict], images: list[str], story: str) -> list[dict]:
    """Match each outline panel with its image and its slice of the story text. """
    sections = {
        int(m.group(1)): m.group(2).strip()
        for m in re.finditer(r"Panel\s+(\d+)\s*:\s*(.*)\s*(?!=Panel\s+(\d+)\s*:\s*|\Z)", story, re.S)
    }
    layout = []
    for panel, image in zip(outline, images):
        text = sections.get(panel["panel"], "")
        caption = re.search(r"Caption:\s*(.*)\s*(?!=Narration:\s*|\Z)", text, re.S)

```

```

narration = re.search(r"Narration:\s*(.*)", text, re.S)
layout.append(
    {
        "panel": panel["panel"],
        "title": panel["title"],
        "scene": panel["scene"],
        "image_prompt": panel["image_prompt"],
        "image": image,
        "caption": caption.group(1).strip() if caption else "",
        "text": narration.group(1).strip() if narration else text,
    }
)
return layout

```

app/exporters.py

```

from datetime import datetime
from pathlib import Path

from fpdf import FPDF

ROOT = Path(__file__).resolve().parent.parent
EXPORTS_DIR = ROOT / "static" / "exports"

FONTS_DIR = ROOT / "static" / "fonts"

def save_pdf(layout: list[dict]) -> str:
    """Write one page per panel to static/exports; return the PDF's URL path."""
    EXPORTS_DIR.mkdir(parents=True, exist_ok=True)
    pdf = FPDF()
    pdf.add_font("DejaVu", "", str(FONTS_DIR / "DejaVuSans.ttf"))
    pdf.add_font("DejaVu", "B", str(FONTS_DIR / "DejaVuSans-Bold.ttf"))
    pdf.add_font("DejaVu", "I", str(FONTS_DIR / "DejaVuSans-Oblique.ttf"))
    for panel in layout:
        pdf.add_page()
        pdf.set_font("DejaVu", "B", 16)
        pdf.multi_cell(0, 9, f"Panel {panel['panel']}: {panel['title']}", new_x="LMARGIN", new_y="NEXT")
        pdf.image(str(ROOT / panel["image"]).lstrip("/"), x=30, w=150)
        pdf.ln(4)
        pdf.set_font("DejaVu", "I", 11)
        pdf.multi_cell(0, 6, panel["scene"], new_x="LMARGIN", new_y="NEXT")
        pdf.ln(2)
        pdf.set_font("DejaVu", "", 11)
        if panel["caption"]:
            pdf.multi_cell(0, 6, "Caption: " + panel["caption"], new_x="LMARGIN", new_y="NEXT")
            pdf.ln(2)
        pdf.multi_cell(0, 6, panel["text"], new_x="LMARGIN", new_y="NEXT")
    name = f"comic_{datetime.now():%Y%m%d_%H%M%S}.pdf"
    pdf.output(str(EXPORTS_DIR / name))
    return f"/static/exports/{name}"

```